PYTHON · MASTER GUIDE

Recursion in Python

A joyful, step-by-step journey from "what is recursion?" to writing it confidently yourself.



Foundations · Call Stack · Traces · Practice

For Absolute Beginners

0 What Recursion Actually Is

The exhaustive foundation before analogies or code

A plain-language definition: Recursion is a way of solving a problem by having a function call **itself**, with a smaller or simpler piece of the original problem, until the piece left is small enough to solve directly without calling anything further.

That's the whole idea. Everything else is detail on top of this.

Why does this even work?

It works because of one key assumption: **if you trust that a smaller version of the problem gets solved correctly, you only need to figure out how to turn that smaller solution into the answer for the current problem.**

> *This is sometimes called the "recursive leap of faith." You don't trace through every single call in your head — you assume the function already works correctly for smaller inputs, and just focus on: given the correct answer to a smaller version, how do I build the answer for my version?*

The two essential ingredients

Every correct recursive function needs exactly two things:

① BASE CASE

The stopping condition — the smallest, simplest version of the problem that can be answered directly, with no further calls. It's the anchor recursion depends on.

② RECURSIVE CASE

Where the function calls itself with an input *closer* to the base case. It breaks the problem into a smaller version of itself, plus a little extra work.

Example: In "sum all numbers from 1 to n," the base case is `n == 0`, because the sum of numbers from 1 to 0 is trivially `0` — no computation needed. The recursive case is `sum(n) = n + sum(n - 1)`: the "extra work" is adding `n`; the smaller problem is `sum(n - 1)`.

The Non-Negotiable Rule: Progress Toward the Base Case

Every recursive call **must** move the input closer to the base case. If it doesn't, the recursion never terminates — the single most common bug beginners write:

```
def broken(n):
    if n == 0:
        return 0
    return broken(n)    # BUG: n never changes! Never reaches the base case.
```

Compare with the correct version:

```
def working(n):
    if n == 0:
        return 0
    return n + working(n - 1)    # n decreases each call — guaranteed progress
```

What happens without a base case?

⚠ **RecursionError** — Python will keep calling the function, over and over, each call waiting on the next. Python limits how many calls can be "waiting" at once (by default, around 1000). Once hit, Python raises:

```
RecursionError: maximum recursion depth exceeded
```

This safety net exists because infinite recursion would otherwise try to consume unlimited memory.

Recursion vs. Iteration

Worth knowing upfront: **anything recursion can do, a loop (iteration) can also do**, and vice versa. Recursion isn't magic — it's a different *style* of expressing repetition, often more natural for problems inherently defined in terms of themselves (factorials, trees, divide-and-conquer). The trade-off: recursion uses more memory (because of the stack) while loops generally don't.

The Anatomy of Every Recursive Function

```
def recursive_function(input):
    if <base_case_condition>: # 1. Are we at the simplest case?
        return <direct_answer> # If so, answer directly.

    # 2. Otherwise, do a little work and call ourselves on a smaller input
    return <combine_work>(recursive_function(<smaller_input>))
```

Ask yourself, every time:

- What is the smallest, simplest case I can answer without recursion? → That's your base case.
- How do I express the problem in terms of a smaller version of itself? → That's your recursive case.

1 The Onion Analogy

Building intuition with a picture you can hold in your head

Imagine an onion 🧅. To get to its core, you peel one layer at a time. Each peel is identical in *action* (peel a layer) but brings you closer to the center. Eventually, there's nothing left to peel — you've hit the core.

RECURSIVE STEP

Peeling one layer and reducing the problem. In code: the part of the function that calls itself again with a smaller input (e.g., n becomes n - 1).

BASE CASE

Reaching the core — the condition that says "stop peeling, we're done." Without it, the function has no reason to stop.

No Base Case = Infinite Peeling. If you never define a core, you'd peel forever. In Python, this shows up as infinite recursive calls, each waiting on the one before it, until Python runs out of memory to track them and raises `RecursionError: maximum recursion depth exceeded`.

Assembling Back Outward

Once you hit the core (base case), the onion analogy has a second half: **assembling back outward**. Each layer you peeled gets "put back" in reverse order — this maps directly to how return values travel back up through each function call, which we cover next.

2 Call Stack & Memory Isolation

The engine underneath every recursive call

What is the Call Stack?

Think of the call stack as a stack of plates. Every time a function is called, Python puts a new plate on top. Every time a function *finishes*, Python takes the top plate off. You can only ever work with the plate on top — the most recently added one. Each "plate" is called a **stack frame**.

Stack Frames Are Isolated

Every time a function calls itself, Python doesn't reuse the same variables — it creates a **brand new, separate stack frame** with its own private copy of local variables. This is the single most important idea in recursion:

> *A variable named n in one frame is a completely different variable from n in another frame, even though they share a name. Changing n in one frame does NOT affect n in any other frame.*

Two Phases of Recursion

WINDING PHASE

Every recursive call pauses the current frame exactly at the call line and pushes a new frame on top. This continues until a base case is reached. Nothing is returned yet — the stack just grows taller.

UNWINDING PHASE

Once the base case hits, its frame returns a value. Python pops it off and feeds that value back into the exact line where the parent was paused — and so on up the stack.

This winding-then-unwinding is exactly the peel-then-reassemble motion of the onion.

3 Traced Code Examples

Watching the stack wind and unwind, line by line

Example 1 — Count Down (work BEFORE the recursive call)

```python
def count_down(n):
    if n == 0:                  # Base case
        print("Liftoff!")
        return
    print(n)                    # Work happens BEFORE the recursive call
    count_down(n - 1)           # Recursive step

count_down(3)
```

Step	Action	Call Stack (top→bottom)
1	count_down(3) called. Prints 3. Calls count_down(2), paused.	`[3]`
2	count_down(2) called. Prints 2. Calls count_down(1), paused.	`[2, 3]`
3	count_down(1) called. Prints 1. Calls count_down(0), paused.	`[1, 2, 3]`
4	count_down(0). Base case. Prints "Liftoff!". Returns None.	`pops`
5	Frame 1 resumes, nothing left to do, returns None.	`pops`
6	Frame 2 resumes, nothing left to do, returns None.	`pops`
7	Frame 3 resumes, nothing left to do, returns None.	`empty`

Output → 3, 2, 1, Liftoff! (printing happens during the winding phase)

Example 2 — Count Up (work AFTER the recursive call, during unwinding)

```python
def count_up(n):
    if n == 0:                  # Base case
        return
    count_up(n - 1)             # Recursive step happens FIRST
    print(n)                    # Work happens AFTER — during unwinding

count_up(3)
```

Step	Action	Call Stack
1	count_up(3) calls count_up(2), paused before print(3).	`[3]`
2	count_up(2) calls count_up(1), paused before print(2).	`[2, 3]`
3	count_up(1) calls count_up(0), paused before print(1).	`[1, 2, 3]`
4	count_up(0). Base case. Returns None immediately.	`pops`
5	Frame 1 resumes at print(n) → prints 1. Returns.	`pops`
6	Frame 2 resumes at print(n) → prints 2. Returns.	`pops`
7	Frame 3 resumes at print(n) → prints 3. Returns.	`empty`

Output → 1, 2, 3 (reverse of Example 1 — printing happens during unwinding)

Example 3 — Factorial (return values flow upward and multiply)

```python
def factorial(n):
    if n == 0:                  # Base case
        return 1
    return n * factorial(n - 1)    # Multiply n by result of smaller problem

print(factorial(4))
```

Frame (winding)	Paused at	Waiting for
factorial(4)	4 * factorial(3)	result of factorial(3)
factorial(3)	3 * factorial(2)	result of factorial(2)
factorial(2)	2 * factorial(1)	result of factorial(1)
factorial(1)	1 * factorial(0)	result of factorial(0)
factorial(0)	base case — returns 1 immediately	

Frame resumes (unwinding)	Computes	Returns
factorial(1)	1 * 1	1
factorial(2)	2 * 1	2
factorial(3)	3 * 2	6
factorial(4)	4 * 6	24

Output → 24 — each frame combines the value handed up from below with its own n.

4 Practice Exercises

Five beginner challenges to build real confidence

1 Fibonacci Sequence

Write a recursive function to find the nth Fibonacci number (0-indexed: 0, 1, 1, 2, 3, 5, 8, ...).

`fibonacci(6) → 8`

Hint: Base case: n==0 → 0, n==1 → 1. Recursive step: fibonacci(n-1) + fibonacci(n-2).

```
def fibonacci(n):
    # Base cases: the first two numbers in the sequence
    if n == 0:
        return 0
    if n == 1:
        return 1
    # Recursive step: sum of the two preceding numbers
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(6))  # Output: 8
```

2 Palindrome Checker

Check if a string reads the same forwards and backwards using recursion.

`is_palindrome("racecar") → True` `is_palindrome("hello") → False`

Hint: Base case: length ≤ 1 is always a palindrome. Recursive step: compare first/last chars, recurse on the inner substring.

```
def is_palindrome(s):
    # Base case: empty string or single character is a palindrome
    if len(s) <= 1:
        return True
    # If outer characters don't match, it's not a palindrome
    if s[0] != s[-1]:
        return False
    # Recursive step: check the "inner" substring
    return is_palindrome(s[1:-1])
```

3 Sum of Digits

Calculate the sum of the individual digits in a positive integer.

`sum_of_digits(1234) → 10`

Hint: Base case: single-digit (n < 10) returns itself. Recursive step: last digit + sum_of_digits(rest).

```python
def sum_of_digits(n):
    # Base case: single-digit number
    if n < 10:
        return n
    # Recursive step: last digit + sum of the rest
    return (n % 10) + sum_of_digits(n // 10)
```

4 Power Function

Calculate base raised to the power of exponent recursively.

`power(2, 5) → 32`

Hint: Base case: exponent 0 → 1. Recursive step: base * power(base, exponent-1).

```python
def power(base, exponent):
    # Base case: anything to the power of 0 is 1
    if exponent == 0:
        return 1
    # Recursive step: multiply base by result of smaller power
    return base * power(base, exponent - 1)
```

5 Reverse a String

Reverse a text string using recursion.

`reverse_string("hello") → "olleh"`

Hint: Base case: empty string reverses to itself. Recursive step: reverse everything after the first char, then append the first char at the end.

```python
def reverse_string(s):
    # Base case: empty string is already "reversed"
    if len(s) == 0:
        return s
    # Recursive step: reverse the rest, then append first character at the end
    return reverse_string(s[1:]) + s[0]
```

✓ You made it through the whole guide — go peel some onions of your own!